

CS102: Data Structures and Algorithms

Week 2: Linear Abstract Data Types & Interaction Models

Milestone 1: Linear Abstract Data Types & Interaction Models

Data structure topologies are governed by explicit constraints regulating how elements enter and exit address memory space. Re-engineering these computational tracks requires evaluating two distinct data flows: LIFO and FIFO.

Lessons 1 & 2: The Stack Paradigm & LIFO Mechanics

The Stack abstract data type operates on a strict Last-In, First-Out (LIFO) protocol. Elements are pushed and popped from a single active memory offset designated as the top of the stack.

■ Expanded Content: Real-World Stack System Implementations

- **Text Editor Undo Engines:** Each discrete character modification or stroke is pushed onto a transaction stack; an undo operation pops the most recent transaction block first.
- **Browser Back Navigation:** Current URI paths are pushed sequentially onto a stack frame; hitting 'Back' pops the top path node to revert to the immediate predecessor state.
- **Backspace Operations:** Deleting string tokens backward from the tail end of an array line mirrors pure stack pop extraction.

```
# Functional Paradigm: Simulating a Bounded LIFO Transaction Stack
transaction_stack = []

# Push Operations: Elements concatenate strictly to the top
transaction_stack.append("mutation_node_01")
transaction_stack.append("mutation_node_02")

# Pop Operation: Extract top node
if transaction_stack:
    active_node = transaction_stack.pop() # Yields mutation_node_02
```

Lessons 7 & 8: The Queue Paradigm & FIFO Turn-Taking

The Queue data structure uses a First-In, First-Out (FIFO) entry protocol. Elements join the structure at a boundary node designated as the back and are extracted exclusively from the front.

■ Expanded Content: System Applications of FIFO Queues

- **Asynchronous Print Spoolers:** Network document payloads are queued in arrival order; the spooler processes jobs from front to back to maintain absolute fairness.
- **Supermarket Checkout / Ticket Lines:** Multi-agent structures that process items strictly in the order they arrive, ensuring fairness.

- **Inventory Refresh Logistics:** Perishable asset systems (e.g., cold storage grocery pipelines) pull out the oldest assets first to prevent expiration losses.

Milestone 2: Python Interface and Complexity Management

Implementing linear collections using standard Python primitives requires analyzing algorithmic execution costs and guarding against runtime memory boundaries.

Lessons 3 & 4: Python List Stacks & Underflow Safeguards

A standard Python list serves as a high-performance stack implementation. `list.append()` handles push actions, and `list.pop()` handles pop operations at the end of the array. To prevent application crashes from Stack Underflow, you must check the array's size before running a pop operation.

```
# Safe Bounded Stack Interface with Underflow Guardrails
def execute_safe_pop(stack_reference):
    if stack_reference:
        return stack_reference.pop()
    return "Underflow Blocked: Stack contains zero elements."
```

Lessons 9 & 10: Array Queues vs. Dedicated Deque Performance

A basic Python list can simulate a queue by pairing `list.append()` with `list.pop(0)`. However, using `list.pop(0)` forces an expensive linear operation because every remaining element must shift one slot left in memory. For heavy workloads, using `collections.deque` offers constant-time efficiency.

Operation Type	Standard Python List Primitive	Optimized <code>collections.deque</code> Element
Enqueue (Back)	<code>list.append()</code> – $O(1)$	<code>deque.append()</code> – $O(1)$ Amortized
Deque (Front)	<code>list.pop(0)</code> – $O(n)$ Heavy Shift	<code>deque.popleft()</code> – $O(1)$ Constant
Underflow Guard	<code>if len(list) > 0:</code>	<code>if deque_reference:</code>

Milestone 3: Composite Topologies & Nested Application Engine

Complex system challenges require nesting abstract data types inside one another, matching diverse business requirements within unified architectural layouts.

Lessons 5 & 12: Nested Topologies — The Dungeon Simulation

The 'Dungeon Simulation' model demonstrates a multi-layered topology: an outer FIFO queue that processes incoming agents in strict arrival order, where each agent holds an internal LIFO stack representing their personal item inventory.

```
# Production Pipeline: Outer FIFO Queue containing Inner LIFO Stacks
from collections import deque

adventurer_line = deque()
agent_aria = {"name": "Aria", "backpack": ["potion", "torch", "key"]}
agent_borin = {"name": "Borin", "backpack": ["bread", "shield"]}

adventurer_line.append(agent_aria)
adventurer_line.append(agent_borin)

while adventurer_line:
    active_agent = adventurer_line.popleft() # FIFO Extraction
    while active_agent["backpack"]:
        extracted_item = active_agent["backpack"].pop() # LIFO Extraction
        print(f"{active_agent['name']} used: {extracted_item}")
```